DevOps 리눅스 실습 보고서

안채림 · 202544049

2026학년도 2학기 중간 종합 평가 · 제출일 2026.10.23

Ubuntu 24.04.4 LTS(WSL2)에서 셸 스크립트와 네트워크를 확인하고, Linux 컨테이너로 방명록 서비스를 실행했다. 주차별 README 4개를 작성했으며, 7주차 48쪽의 web=3 유지 문제는 Bash 반복 실행으로 해결했다.

1. 공개 GitHub 저장소

https://github.com/cheacheada/DevOps
로그인하지 않은 브라우저에서 Public 표시와 main의 주차별 디렉터리를 확인했다. 기존 수업 저장소를 계속 사용했다.

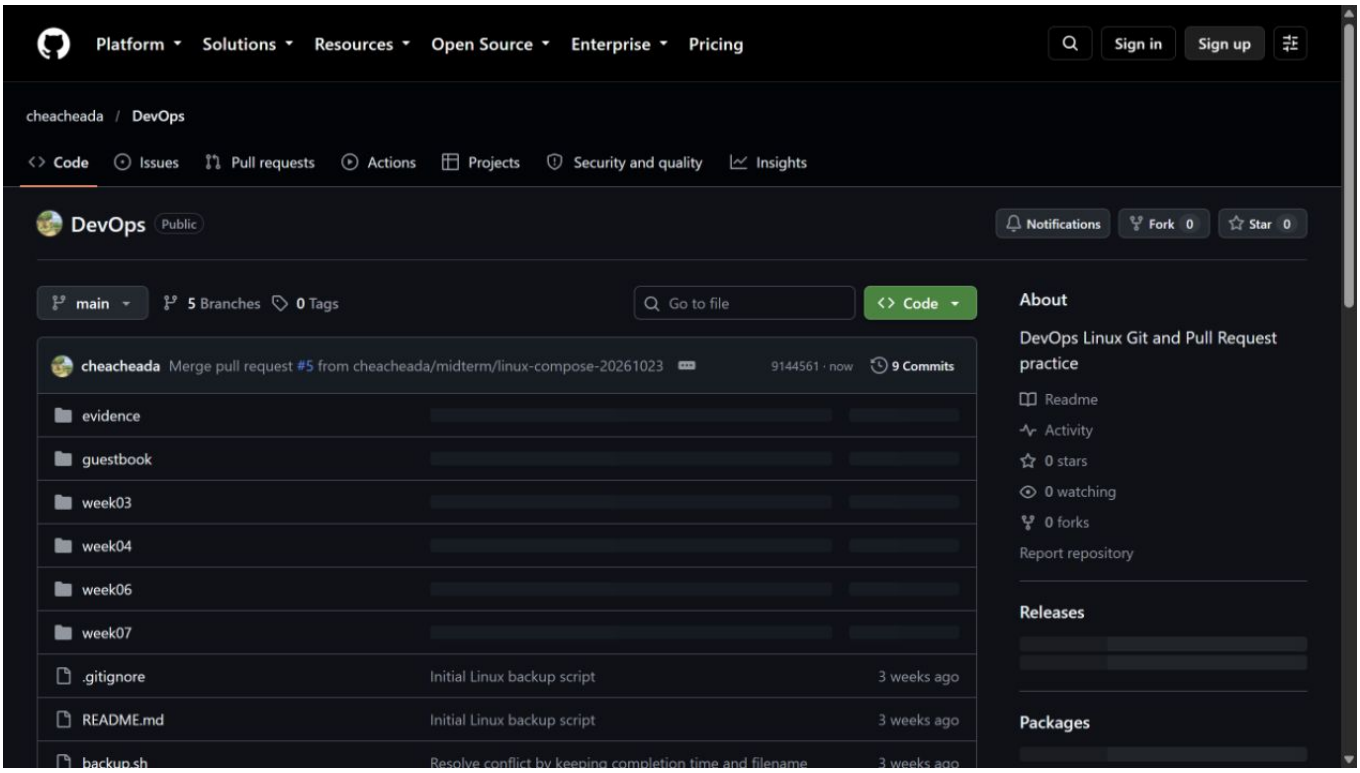


그림 1. 실제 공개 저장소 화면. PR #5 병합 후 week03·04·06·07 및 guestbook이 main에 반영됐다.

평가 항목	제출 근거
저장소 공개 (20점)	공개 주소 및 위 실제 캡처
최소 3개 주차 README (30점)	3·4·6·7주차 총 4개 작성
Issue부터 Merge 설명 (30점)	별도 PPT 1장, Issue #4 및 PR #5
web=3 유지 (20점) 및 가산점	Bash 코드, 중지·복구 검증, 장점·단점·한계

2. 주차별 학습 정리: 셸과 네트워크

3주차 · 셸 스크립팅

반복 작업을 Bash 스크립트로 만들면 같은 순서를 다시 실행할 수 있다. shebang은 해석기를 지정하고 실행 권한은 chmod로 부여한다. 인자가 없을 때 기본 경로를 사용하되, 경로가 디렉터리인지 먼저 검사하도록 백업 스크립트를 작성했다.

```
cd week03
chmod u+x backup.sh
bash -n backup.sh
./backup.sh sample
tar -tzf "$(ls -t backups/*.tar.gz | head -1)"
```

실제 결과: sample/note.txt를 포함한 압축 파일이 생성됐다. 존재하지 않는 경로는 오류 메시지와 종료 코드 1로 거부했다. 명령이 실행됐다는 사실뿐 아니라 결과물 내용까지 확인해야 한다는 점을 배웠다.

4주차 · 네트워크

IP는 호스트를 식별하고 포트는 서비스를 구분한다. DNS는 이름을 IP로 해석한다. 리눅스에서 HTTP 서버를 열고 curl과 ss로 응답과 리스닝 포트를 확인했다. 같은 포트에 두 번째 서버를 실행해 충돌도 재현했다.

```
python3 -m http.server 18080 --bind 127.0.0.1
curl -I http://127.0.0.1:18080
ss -lntp | grep ':18080'
# 실제 결과 발췌
HTTP/1.0 200 OK
LISTEN ... 127.0.0.1:18080 ... python3
OSError: [Errno 98] Address already in use
```

포트 충돌은 리스닝 중인 프로세스를 확인하거나 다른 포트를 선택해 해결한다. 이 원리는 web 3개가 동일한 호스트 8080을 함께 사용할 수 없는 이유와 연결된다. Compose에서는 nginx만 8080을 공개하고 web은 내부 5000 포트로 통신하도록 구성했다.

README 경로: week03/README.md, week04/README.md
실행 원문: evidence/linux-checks.txt
강의 연결: 제공된 2주차.pdf 표지의 3주차 셸 스크립팅, 4주차.pdf 네트워크. 추가자료를 참고해 *:Zone.Identifier도 Git에서 제외했다.

3. Dockerfile과 Compose

6주차 · Dockerfile

Dockerfile은 이미지를 만드는 절차를 코드로 기록한다. 수업 제공 방명록의 python:3.12-slim Linux 이미지를 사용했다. 의존성 목록을 먼저 복사하고 설치한 뒤 코드를 복사하면 코드 수정 시 설치 레이어를 재사용하기 쉽다.

명령	내가 이해한 역할
FROM / WORKDIR	기반 이미지와 작업 디렉터리 지정
COPY / RUN	파일 복사와 빌드 시 의존성 설치
USER / ENV	일반 사용자와 기본 환경변수 설정
EXPOSE / CMD	포트 정보 선언과 시작 명령 지정

실제 빌드 후 컨테이너 내부 id 결과는 uid=1000(appuser)였다. EXPOSE는 포트 공개 명령이 아니므로 외부 연결은 Compose의 ports로 지정했다. 메모리 방식은 프로세스가 종료되면 데이터가 사라지므로 다음 실습에서 Redis로 저장소를 분리했다.

7주차 · Compose

nginx 1개, web 3개, Redis 1개를 하나의 Compose 프로젝트로 관리했다. nginx는 localhost:8080의 요청을 받고 Docker 내부 DNS의 web:5000으로 전달한다. web은 REDIS_HOST=redis로 공통 저장소에 접근한다. Redis와 web의 healthcheck로 준비 상태를 확인했다.

```
cd guestbook
cp .env.example .env
# 비밀번호를 바꾼 뒤 실행 (.env는 Git 제외)
docker compose config --quiet
docker compose up -d --build --scale web=3 --wait
docker compose ps
```

12번의 /healthz 요청에서 서로 다른 hostname 3개를 확인했다. down 후 up해도 방명록 글이 남았다. 별도 테스트 프로젝트에서는 down -v 이후 키 존재 여부가 0이 되어 볼륨 삭제 차이도 확인했다. 호스트명 변경은 요청 분산의 근거이며 균등 분배나 무중단을 보장하는 근거는 아니다.

README: week06/README.md, week07/README.md. 실행 원문: evidence/docker-checks.txt. 수업 코드 기반 실습이며 GHCR 업로드는 수행하지 않았다.

4. Issue부터 Merge까지

문제는 Issue #4에 정리했다. 작업 브랜치에서 코드와 문서를 수정하고 검증 결과를 PR #5에 적었다. PR 설명의 Closes #4로 Issue를 연결했으며 main 병합 후 Issue 종료를 확인했다. 아래 흐름은 별도 PPT 한 장으로도 제출한다.

Issue부터 Merge까지

내가 해결할 문제를 기록하고, 검증한 변경을 main에 합치는 과정

01 Issue로 목표 정리

web 하나를 멈춰도 다시 3개가 되도록 문제와 완료 조건을 Issue #4에 적었다.

02 작업 브랜치 준비

main에서 별도 브랜치를 만들었다. midterm/linux-compose-20261023에서 작업했다.

03 코드 수정과 Commit

Compose와 Bash 감시 스크립트를 작성했다. 변경 내용을 확인하고 commit으로 묶었다.

04 Linux에서 검증

web을 중지해 2개가 된 것을 확인했다. 감시기 실행 후 3개로 복구되는지 확인했다.

05 Pull Request로 비교

브랜치를 push하고 PR에서 main과 비교했다. 검증 결과와 한계를 적고 Closes #4로 연결했다.

06 Merge 후 최종 확인

PR을 main에 합친 뒤 파일과 기록을 확인했다. Issue 종료와 공개 README 링크도 확인했다.

사례: cheacheada/DevOps · Issue #4 · Linux Compose 복구 실습

그림 2. Issue부터 Merge까지 설명한 PPT 1장. 텍스트는 PPT에서 편집 가능하다.

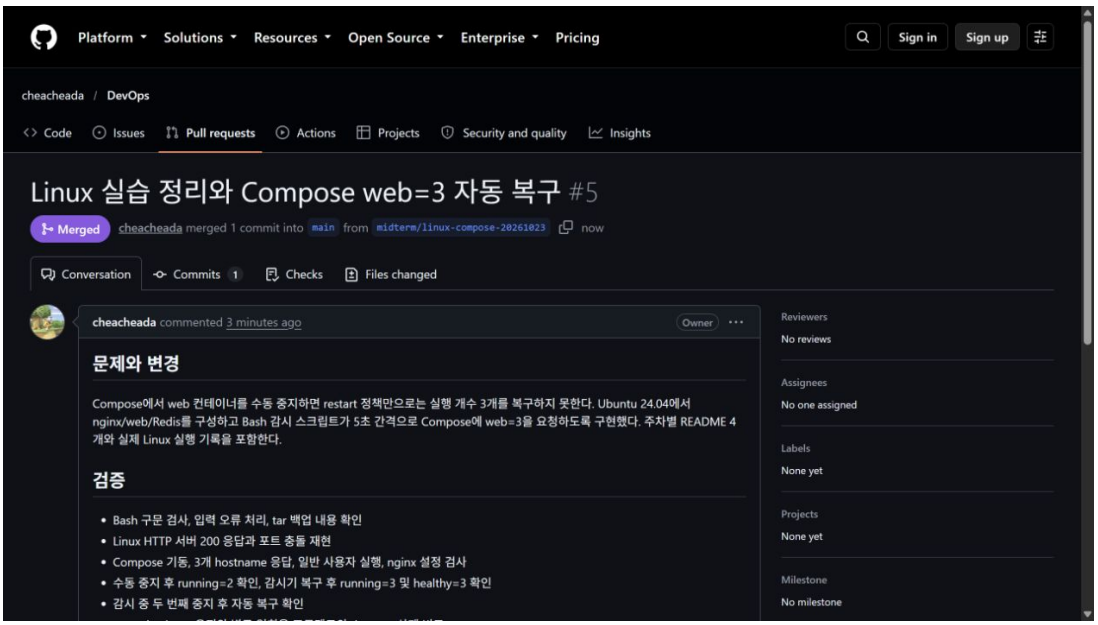


그림 3. PR #5의 실제 Merged 표시와 main 병합 기록.

Issue #4 / Pull Request #5
브랜치: midterm/linux-compose-20261023
이번 PR에서는 병합 충돌이 발생하지 않았다. 충돌이 있다면 충돌 파일을 수정한 뒤 검증·commit·push하고 다시 병합해야 한다.

5. web=3 유지 방법과 Linux 코드

48쪽에서는 docker stop으로 중지한 web이 자동으로 돌아오지 않는다. unless-stopped는 수동 중지를 존중하기 때문이다. restart를 always로 바꾸어도 수동 stop 직후 재시작을 보장하지 않는다. Compose scale도 실행 시점에 개수를 맞추므로 반복 실행이 필요하다.

제안 방법: Bash 감시기

먼저 전체 서비스를 정상 기동한다. 그다음 감시기가 5초 간격으로 Compose에 web=3 상태를 요청한다. 부족하면 생성하고 중지된 컨테이너는 다시 시작한다. 정상 컨테이너의 불필요한 재빌드·이미지 다운로드는 막는다. 다음은 제출 코드의 핵심 부분이다.

```
#!/usr/bin/env bash
set -Eeuo pipefail
cd -- "$(dirname -- "${BASH_SOURCE[0]}")"
exec 9>.reconcile.lock
flock -n 9 || exit 1
trap 'exit 0' INT TERM
docker compose config --quiet
while true; do
  if docker compose up -d --no-deps --no-build ₩
      --pull never --scale web=3 web; then
    echo "$(date -Iseconds): reconciliation done"
  else
    echo "reconcile failed; retry later" >&2
  fi
  sleep 5 & wait $!
done
```

전체 코드는 guestbook/reconcile-web.sh에 있다. 전체 버전은 감시 간격 유효성 검사와 실행 개수 기록, ONCE 단일 실행 옵션을 포함한다. 실행 예시는 다음과 같다.

```
# 터미널 A
cd guestbook
bash reconcile-web.sh
# 터미널 B
id=$(docker compose ps -q web | head -1)
docker stop "$id"
sleep 8
docker compose ps
```

flock은 같은 디렉터리에서 감시기가 중복 실행되는 것을 막는다. 정비나 전체 종료 시에는 감시기 터미널에서 Ctrl+C를 먼저 누른다. Docker 접근 권한이 있는 Linux 사용자로 실행해야 한다.

6. 실제 복구 검증 결과

실행 환경: Ubuntu 24.04.4 LTS, WSL2 x86_64, Bash 5.2.21, Docker Engine 29.1.3, Docker Compose 2.40.3. 2026년 10월 23일 Linux 환경에서 직접 실행한 결과다.

실험	관찰 결과
감시기 없이 web 하나 수동 중지	6초 뒤에도 running=2
감시기 실행	8초 대기 후 running=3
감시 중 동일 web을 다시 중지	자동 복구 PASS, running=3
준비 상태 추가 확인	web 3개 모두 running healthy
요청 분산	12회 응답에 hostname 3개 관찰
일반 down/up	Redis 방명록 count=1 유지
별도 프로젝트 down -v/up	Redis proof 키 exists=0

실제 로그 발췌

```
Running before recovery=2
Running after recovery=3
Automatic second recovery: PASS (running=3)
/devops-midterm-web-1 running healthy
/devops-midterm-web-2 running healthy
/devops-midterm-web-3 running healthy
[2026-10-23T12:23:05+09:00] running_web=3 target=3
```

실험에서 8초를 기다린 뒤 복구 상태를 확인했으며, 복구에 정확히 8초가 걸렸다는 의미는 아니다. 실제 복구 시간은 검사 간격과 Compose 실행·컨테이너 시작 시간의 영향을 받는다. nginx DNS 캐시와 재시작 시점 때문에 일시적인 요청 실패가 생길 수 있다.

running과 healthy의 구분

running은 프로세스가 실행 중이라는 뜻이다. healthy는 헬스체크가 성공했다는 뜻이다. 감시기는 개수를 복구하며, 이미 실행 중이지만 unhealthy인 web을 자동 교체하지 않는다. 최종 검증에서는 up --wait와 inspect로 세 개의 healthy도 별도로 확인했다.

원문 파일: evidence/recovery.txt, evidence/reconciler-output.txt, evidence/docker-checks.txt. 로그는 실제 출력이며 보기 편하도록 발췌했다.

7. 장점·단점·한계와 참고자료

장점

수업에서 배운 Bash 반복문, 종료 코드, 프로세스, Docker Compose 명령만으로 구현할 수 있다. 스크립트가 짧아 동작을 설명하기 쉽고, 로그로 개수 복구를 확인할 수 있다. 별도의 클러스터 구성 없이 한 대의 Linux에서 재현할 수 있다.

단점

검사 주기만큼 복구가 늦고, 매 주기 Compose 명령을 실행하는 비용이 든다. 사용자가 정비를 위해 web을 멈추더라도 감시기가 다시 켜므로 감시 종료 절차가 필요하다. 중복 실행은 flock으로 막지만 다른 경로·프로젝트에서 별도 감시기를 실행하는 운영 실수까지 막지는 못한다.

한계와 개선 방향

이 방법은 순간적으로도 반드시 3개라는 보장이 아니라, 장애 후 목표 상태인 3개로 수렴시키는 방식이다. 호스트·Docker 데몬·감시 프로세스가 멈추면 복구하지 못한다. unhealthy 교체, Redis 자체 장애 복구, 여러 호스트로 분산, 무중단 배포는 해결하지 않는다. 운영 환경에서는 상태 관찰과 복구 정책을 갖춘 오케스트레이터를 검토해야 한다. 단일 서버 실습 답안과 운영 환경의 고가용성은 구분해야 한다.

학습 소감

명령 한 번으로 컨테이너를 3개 띄우는 것과 이후에도 3개를 유지하는 것은 별개의 문제였다. 수동 중지 실험으로 restart 정책의 범위를 이해했고, 셸에서 배운 반복 실행을 Compose에 연결할 수 있었다. 개수뿐 아니라 준비 상태와 데이터 유지도 함께 확인해야 서비스 동작을 설명할 수 있다는 점을 배웠다.

참고자료 및 재현 파일

수업 제공 자료: 2주차.pdf(표지상 3주차 셸), 3주차.pdf(자동화·협업), 4주차.pdf, 4주차 추가자료.pdf, 5주차.pdf, DevOps_06_도커파일.pdf, DevOps_07_도커_컴포즈.pdf(특히 43-48쪽, 61쪽). 주차별 최소 개수 조건에 맞춰 3·4·6·7주차를 정리했다.

Docker 공식 문서: Start containers automatically
Docker 공식 문서: docker compose up
확인일: 2026.10.23. restart 수동 중지 동작과 --scale, --wait 의미를 확인했다.

코드: guestbook/compose.yaml, nginx.conf, reconcile-web.sh
학습 기록: week03·04·06·07/README.md
별도 제출: Issue-to-Merge.pptx (1장)
작성 지원: 강의 제공 코드와 자료를 토대로 AI 도구로 정리·구현했으며 실제 실행 로그를 근거로 검증했다.